

Neurons are typically introduced into an LNN in an Unknown state, with allowance for rule or fact truth values to be directly assigned if known. When offered new bounds at inference, lower bounds monotonically tighten upward and upper bounds monotonically tighten downward.

F.3 Definition

One of the key insights of the tailored activation function approach is to define dynamic False, Fuzzy and True regions in the domain. Both approaches define static regions in the range. The constrained approach imposes static regions on the domain, which is why constraints become necessary.

The tailored activation function rather keeps track of the dynamic semantically-derived boundary points between the regions (i.e. from the classical truth table). The boundary between False and Fuzzy is called x_F and the boundary between Fuzzy and True is called x_T . Then the tailored activation function is required to be monotonic and to go through these two dynamic points $(x_F, 1 - \alpha)$ and (x_T, α) . Monotonicity is not a new requirement and neither is the requirement of going through two points. What is new, is that the two points are dynamic and that the classically True region is more than a single point. This latter advantage is significant, since smooth activation functions such as the sigmoid function may be used without worrying about reaching a maximally true value, i.e. 1, to represent classically true ($\alpha < 1$).

The biggest advantage is that there are no constraints on the weights; the activation function, by definition, adjusts as the weights change, maintaining the logical semantics. The second major advantage is that there are useful gradients everywhere.

The price to pay for this, of course, is that these truth points depend on the weights of the operands, therefore every node requires its own activation function.

Weighted inputs to the activation function are neural-network-like, in that they require a dot product of weight and bounds vectors of an n -ary input, $f_w(\mathbf{w} \cdot \mathbf{x})$.

In total there are four points of interest:

$$(x_{min}, y_{min}) = (\beta - \sum_{i \in I} w_i, 0) \quad (90a)$$

$$(x_F, y_F) = (\beta - \alpha w_{max}, 1 - \alpha) \quad (90b)$$

$$(x_T, y_T) = (\beta - \sum_{i \in I} w_i \cdot (1 - \alpha), \alpha) \quad (90c)$$

$$(x_{max}, y_{max}) = (\beta, 1) \quad (90d)$$

The monotonicity requirement does produce one non-trivial constraint related to the weights in combination with α , namely that $x_F \leq x_T$. Equality can be optionally ruled out, $x_F \neq x_T$ if a continuous activation function is desired. This ensures that there is a non-empty Fuzzy region interpolating between the regions of classicality. Enforcing this constraint, is easily achieved from the outset by choosing an appropriate α :

$$\frac{\sum_{i \in I} w_i}{\sum_{i \in I} w_i + w_{max}} < \alpha \leq 1 \quad (91)$$

The constraints of $x_{min} \leq x_F$ and $x_T \leq x_{max}$ are automatically enforced by definition.

$$\forall i \in I, \quad 0 \leq w_i \quad (92)$$

$$\forall i \in I, \quad w_i \leq 1 \quad (93)$$

Note, however, that α has lower bound $n/(n + 1)$ for n the largest number of operands of any connective in the knowledge graph. To maximise learning gradients, this global α can therefore be initialised to $\alpha = \frac{1}{2}$ (the maximum classical range) and update alpha accordingly.

As an extension, it may be useful to have an α learnable per node, allowing each node to keep a relative interpretation of the truth. This would then require that the constraint (91) be enforced at every parameter update, though the exploration of this is future work.

The required constraints (92) can be achieved by clamping at zero⁵. There are other optional orthogonal constraints on the weights, (93), that may be useful for simplifying interpretability, which we do employ: scaling all input weights by w_{max} — giving some effect to updates calling for an increase beyond one. Alternatively, we may simply clamp at zero and one — ignoring gradients that take weights out of the range.

An analysis of the tailored activation function, reveals that the output is independent of β . Therefore it can be eliminated or for convenience we analytically define it as:

$$\beta_C = \sum_{i \in I} w_i$$

simplifying (90) to:

$$(x_{min}, y_{min}) = (0, 0) \tag{94a}$$

$$(x_F, y_F) = (\beta_C - \alpha w_{max}, 1 - \alpha) \tag{94b}$$

$$(x_T, y_T) = (\sum_{i \in I} w_i \cdot \alpha, \alpha) \tag{94c}$$

$$(x_{max}, y_{max}) = (\beta_C, 1) \tag{94d}$$

This simplification anchors $(x_{min}, x_{max}) = (0, \beta_C)$ while weights fluctuate and ensures functional inverses behaves equivalently $(y_{min}, y_{max}) = (0, \beta_C)$.

F.4 Satisfying the classical truth table

F.4.1 Design choice concerning x_F

Both the weighted fuzzy logic and tailored activation approaches identify the boundaries in a similar way, namely, by consulting the classical truth table. However, having introduced weights to operands, there is an interesting design choice that arises: when weights are equal and truth values are classical, the classical truth table should be perfectly obeyed, but when the weights are unequal there is a choice as to where to place the boundary. Should classicality still be imposed? For example, the False-Fuzzy boundary point, x_F , of conjunction could be defined by insisting that the classical truth table hold when the lowest weighted operand is `False` and the rest `True` (i.e. when w_{min} 's operand is `False` — derived from 86). But then the undesirable behaviour of a very low weighted operand having undue influence is observed. Rather, it is proposed to pay attention to only the largest weight, w_{max} .

Firstly, it is true that non-classical behaviour is exhibited for a `False`, lowly weighted input, but then this is desirable. In the constrained approach, this is achieved by learning slacks to switch off constraints or not, Equation 86*. In the tailored activation approach slacks are not needed and the proposed choice of w_{max} forces non-classicality. This fulfils the expectation of what a low weight means, in spite of fewer parameters to over-fit. This is also good for interpretability, even if it suffers a slight reduction in performance. Afterall, LNN's are supposed to be logically-based and if formulae need to be augmented, it is aesthetically more appropriate to do so logically rather than with extra uninterpretable parameters.

Secondly, the tailored-activation approach provides a second justification for w_{max} , namely the correct behavior when weights go to zero. Indeed for an n -ary conjunction, if all but one weight goes to zero, the identity function is recovered for the remaining operand. When all operands have weights equal to zero, the operator is effectively removed from its parent's formula — illustrated in Figure 5.

⁵though allowing weights to become negative may be useful as a form of negation of the operand. This, however, would come with extra complications. Getting this right in future work would have many advantages, including a generic neuron that can learn many gate configurations — equivalent to a logical operation — solely from the weights

F.4.2 Classical truth tables of other logical operations beside conjunction

The procedure of choosing the boundary points of the tailored activation approach to reflect the classical truth table could be repeated for each logical operation. However, it is more elegant to simply rely on the universality result of classical logic, namely that the set consisting of the unary Not and the binary And gates is universal for classical binary computation.

Through universality everything behaves as expected on classical inputs: $(x \rightarrow y) = (\neg x \oplus y) = \neg(x \otimes \neg y)$, and $((x \rightarrow 0) \rightarrow 0) = x$. The properties of $(x \otimes x) = (x \oplus x) = x$, also hold for classical inputs.

This is contrast to the constrained approached, where implication is defined via the residuum and not the logical definition. The residuum procedure is a sufficient (though not necessary) condition to ensure upward modus-ponens operates as expected. With the tailored activation approach, since classicality is perfectly captured, upward modus-ponens for classical values is guaranteed to work. The expected behaviour for non-classical values must be checked and indeed it is also respected.

F.4.3 Design choice concerning arity

The above discussion about universality would suggest that the n -ary conjunction should simply be decomposed into nested binary conjunctions, thereby retaining all the familiar classical properties. However, there is no unique way of performing the decomposition and an n -ary form of the tailored-activation function is easily definable. This, then becomes a design choice that must be made, namely when encountering an n -ary conjunction, how should it be represented?

The disadvantage of the n -ary form is that it is not equivalent to the decomposed form, which is the form that is guaranteed to behave classically. Nevertheless, it is the form we experiment with, since it results in a simpler logical tree.

F.5 Alpha interpretation – size of classical regions

The choice of hyper-parameter (α) is an interpretable design decision that provides useful functionality for the tailored activation function. It defines the size of the classical regions and influences the rate of learning.

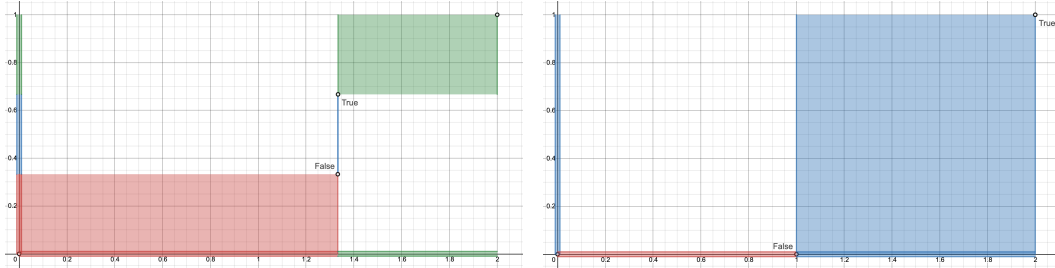


Figure 2: Regions of classicality as alpha varies; left - minimum alpha, right - maximum alpha

At the extrema of equation (91), the choice of alpha defines where the database designer places their trust — the facts or the rules. At its maximum ($\alpha = 1$), complete trust is placed in the database rules, with flat gradients in the classical regions ensuring that rules remain immutable regardless of the truth of classical input facts. This may be of interest to regulated environments, where the preservation of rules are demanded. At its minimum ($\alpha > n/(n + 1)$), the designer trusts the facts over the rules. This may apply for machine generated rules that are noisy at best and incorrect at worst. Here, the LNN will maximise the gradients and quickly learn to correct rule weights. Reducing alpha further reduces the Fuzzy region and increases the range of classicality, highlighted in blue in figure 2.

An alpha between the extrema will allow for variable learning, optionally with a global alpha or an alpha per node, where learning as fast as possible further requires node dependent alphas.

A global alpha places constraints on the weights at initialisation, constrained by the number of input arguments, whereas an alpha node requires constraints to be enforced for each weight update. An alpha per node additionally requires a rescaling of alphas between sub-formula since the relative